WEEK 10

Targeted Python review

Targeted optional Python review

Revisit a known topic only where needed

State, aliases and repeatable runs

Names may share an object

A second name does not create fresh state.

```
items = []; alias = items
alias.append(1); print(items)  # [1]
```

Repeated actions see current state

A retry may repeat its effect rather than reset it.

```
count = 0; count += 1
count += 1; print(count)  # 2
```

A fresh copy is separate

A shallow copy separates the outer list, not nested objects.

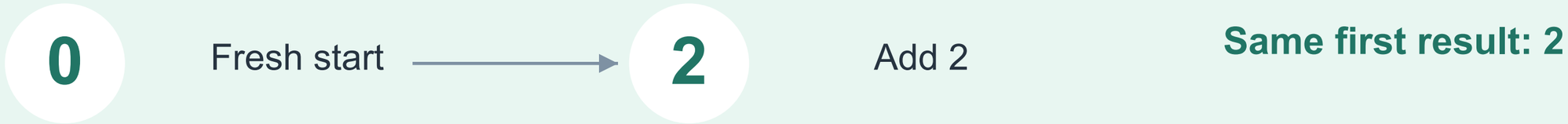
```
items = [1]; copy = items.copy()
copy.append(2); print(items)  # [1]
```

A fresh counter is not a repeated action

Same counter: repeat an action



Independent counter: repeat the starting conditions



Same first result: 2

A reset repeats the starting conditions. A retry uses the current state.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
if __name__ == "__main__":	Choose direct startup	Keep imports reusable
parse_args(...)	Parse command text	Convert and validate inputs
python -m unittest discover	Run a test suite	Use the project test location
try / except ValueError	Observe a refusal	Check state and recovery

Python Basics · sections 08, 10, 15, 16, 17 · use only when you need a lookup.

Optional review: fresh state versus another name

```
01  first = {"count": 0}
02  alias = first
03  separate = {"count": 0}
04  first["count"] += 1
05  print(first["count"], alias["count"], separate["count"])
```

1. Optional handoff reference.
2. Two names can share one object.
3. Use only if it solves a real issue.

Optional review only; keep the final assessments in scope.

Use a guarded main without hiding the calculation

```
01 def planned_portions(ppc, containers):
02     return ppc * containers
03
04 def main():
05     print("Planned portions:", planned_portions(3, 4))
06
07 if __name__ == "__main__":
08     main()
```

1. Direct run prints twelve planning portions
2. Import makes the function available without that print
3. The full CLI validates its external inputs

Import defines reusable functions without starting the plan.

Trace command text into a typed planning count

```
01 import argparse
02 parser = argparse.ArgumentParser()
03 parser.add_argument("--containers", type=int)
04 args = parser.parse_args(["--containers", "3"])
05 assert args.containers == 3
06 print("Planned portions:", 3 * args.containers)
```

1. Predict the parsed type and the planning output
2. many is a parsing failure
3. 0 is a separate domain rejection

Parsing an integer is separate from accepting its domain value.

Give failure an observable process result

Practice argument	Observation	Status
--containers 3	Planned portions: 9 on stdout	0
--containers 0	Positive-count error on stderr	2
--containers many	Integer-parsing error on stderr	2

Exit status and stderr describe failure; they do not decide recovery.

Compare a clean run with a repeated action

```
available = 5
reserved = 0
for attempt in (1, 2):
    available -= 2
    reserved += 2
    print(attempt, available, reserved)
assert (available, reserved) == (1, 4)
# No collection or delivery event was recorded.
```

1. Predict availability after each loop iteration
2. Fresh independent runs each start from 5
3. Do not equate a repeated reservation with delivery

Repeating a write can repeat its effect on existing state.

Make a rejection preserve its caller state

```
def reserve_count(available, requested, reviewed):
    if not reviewed or requested > available:
        raise ValueError("review or availability missing")
    return available - requested

available = 5
try:
    available = reserve_count(available, 2, False)
except ValueError:
    pass  # Deliberate rejection.
assert available == 5
```

1. Controlled example uses positive integer counts
2. The database needs separate rollback evidence
3. No rejected request may gain an event

A failed function call cannot supply a new state value.

Hands-on practice

Short Python rehearsal → volunteer-led project demonstration

Python practice

Choose one real issue

Use only the rehearsal that helps your handoff. The five options are not extra assessment.

Write and verify

Write a small test or helper yourself. Keep its predicted and observed results.

Protect the demonstration

Stop rehearsal before the full operator demonstration and independent verification.

Project studio

Final project · individual verification and handoff

The project need

1

Need

A classmate needs to run the feature using your guide.

2

Operation

Let a peer run your feature; then explain and adapt it independently.

3

Result

The next operator can repeat the result using your instructions.

Your classmate runs it and explains the result.

Document the experiment another person must repeat

1

Environment

Identify interpreter, notebook and required support files.

2

Starting state

Name the CSV fixture and database reset step.

3

Actions

Give the command or ordered cells for the approved feature.

4

Observations

Compare outputs and complete relevant state with predictions.

Inputs, actions and expected state belong in the handoff.

Use a fresh input to demonstrate understanding

ILLUSTRATIVE WORKPLACE CASE

One reviewed pickup request changes before processing. Predict its effect from a clean fixture rather than editing a final report number.

Predict

Write intermediate portions, label and expected state changes.

Run

Rebuild the fixture and execute the stated variation.

Explain

Compare actual evidence and one behavior that stays unchanged.

Trace the condition, its state effect and unchanged event evidence.

Individual verification: explain your own work

Supplied starting point

Integrated baseline and the operator-demo / project-task notebook sections are supplied.

You implement / do

Give reset, run and recovery steps; let a peer operate the feature. During individual verification, predict a fresh variant before executing it.

Check the evidence

Complete the Session 2 operator demo and Session 3 individual verification. Show actual outputs, recovery, a peer finding and your own reasoning.

Show what records establish; keep reservation, collection and delivery distinct.

Project checks

Step	What to show
Implementation	Peer operates; then explain a fresh variant independently.
Checks	Actual output, recovery, limitation and a revised guide.

Show what records establish; keep reservation, collection and delivery distinct.

Further reading in English

1. ## Jupyter Notebook: Running the notebook

   https://jupyter-notebook.readthedocs.io/en/stable/notebook.html

2. ## Google SRE: Postmortem Culture

   https://sre.google/sre-book/postmortem-culture/

Read one relevant section, then bring an example and a question.